

UNIDAD 5 · SEMANA 7

Tuberías y composición de procesos

📅 Semana 7 · fechas por confirmar </> Laboratorio TD4 (resuelto + desafío IA)

~ 2 animaciones

MANUAL M5 · TEORÍA DE LA UNIDAD

Tuberías y composición de procesos

Lectura base de la unidad 5 (≈ 1 h). Las salidas son reales: programas del TD4 ejecutados en el equipo del curso.

CONTENIDO

- | | | | |
|----|---|----|--|
| §1 | La tubería: un búfer con dos extremos | §3 | Redirigir con <code>dup2</code> y ejecutar |
| §2 | Las reglas: bloqueo, fin de archivo y SIGPIPE | §4 | Tuberías de N comandos |
| | | §5 | Más allá |

§1 La tubería: un búfer con dos extremos

`pipe(fds)` crea en el núcleo un búfer circular y devuelve dos descriptores: `fds[0]`, el extremo de **lectura**, y `fds[1]`, el de **escritura**. Lo que se escribe en uno se lee por el otro, en orden, como un flujo de bytes sin fronteras de mensajes. La tubería no tiene nombre (a diferencia de una FIFO, unidad 3): solo pueden usarla los procesos que heredan sus descriptores, así que se crea **antes** del `fork`. Después, cada proceso **cierra el extremo que no usa**: el padre escritor cierra `fds[0]`, el hijo lector cierra `fds[1]`.

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a child
 (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the pipe
 capacity)
 * the parent blocks on write() once the pipe is full. When the child exits, nobody can
 read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken pipe", status
 141
 * in the shell). That only works if the parent CLOSED its own read end: with
```

```

* -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a potential
reader,
* the kernel never raises SIGPIPE and the parent blocks forever.
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end, fds[1] write
end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close");    /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

    /* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");        /* THE line that exercise 2 is
about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n", count);
    exit_if(close(fds[1]) == -1, "close");

```

```
exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
return EXIT_SUCCESS;
}
```

Salida real

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

Practica esto → TD4 · 1

§2 Las reglas: bloqueo, fin de archivo y SIGPIPE

- **Leer** de una tubería vacía bloquea hasta que llegan datos... salvo que ya no quede **ningún** descriptor de escritura abierto en ningún proceso: entonces `read` devuelve 0, el fin de archivo.
- **Escribir** en una tubería llena bloquea hasta que alguien lea. La capacidad por omisión en Linux es 64 KiB (65 536 bytes); se consulta y cambia con `fcntl(F_GETPIPE_SZ / F_SETPIPE_SZ)`.
- Escribir cuando ya no queda **ningún lector** produce la señal `SIGPIPE`, que por omisión mata al proceso (estado 141 = 128 + 13 en el shell); si se ignora, `write` falla con `EPIPE`.
- Una escritura de hasta `PIPE_BUF` = 4096 bytes es **atómica**: no se intercala con las de otros escritores.

Con el TD4 · 2 se ven las tres primeras reglas a la vez. El hijo lee una sola vez 500 bytes y termina; el padre escribe N mensajes de 12 bytes. Si el padre termina antes de que el hijo salga, todo va bien; si el hijo sale primero, el siguiente `write` del padre recibe `SIGPIPE`. Con muchos mensajes esto ocurre siempre, pero en el medio es una **carrera** entre dos procesos que corren en núcleos distintos: el resultado cambia de una ejecución a otra. Medido 30 veces por tamaño:

Salida real

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses; done
 50 messages: 30 x status 0    0 x status 141 (SIGPIPE)
100 messages: 30 x status 0    0 x status 141 (SIGPIPE)
200 messages: 28 x status 0    2 x status 141 (SIGPIPE)
300 messages: 19 x status 0   11 x status 141 (SIGPIPE)
400 messages:  5 x status 0   25 x status 141 (SIGPIPE)
500 messages:  5 x status 0   25 x status 141 (SIGPIPE)
1000 messages: 0 x status 0   30 x status 141 (SIGPIPE)
```

Si el padre olvida cerrar `fds[0]`, él mismo cuenta como lector: nunca recibe `SIGPIPE`, llena los 64 KiB y se bloquea para siempre esperando que "alguien" lea:

Salida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by
timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Practica esto → A5.2 capacidad y carrera · TD4 · 2

§3 Redirigir con `dup2` y ejecutar

`dup2(fds[1], STDOUT_FILENO)` hace que el descriptor 1 apunte a la tubería (unidad 3: copia el puntero a la entrada de la tabla de archivos abiertos). Como `exec` conserva los descriptores (unidad 4), el programa que se ejecute después escribirá en la tubería sin saberlo. Así se conectan programas ajenos, como `ps`, a un lector propio:

C tds/td4/upper.c

```
int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {
        /* child */
        exit_if(close(fds[1]) == -1, "close"); /* otherwise it never sees end of file */
        /*
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
        */

        /* parent */
        exit_if(close(fds[0]) == -1, "close");
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
        exit_if(close(fds[1]) == -1, "close"); /* 1 already refers to the pipe */
        execlp("ps", "ps", (char *)NULL); /* "ps" twice: file to run, then
        argv[0] */
        exit_if(1, "execlp ps");
        return EXIT_FAILURE;
    }
}
```

Salida real

```
$ ./upper | head -4
PID TTY          TIME CMD
6706 ?           00:00:25 UVICORN
9771 ?           00:00:03 SYSTEMD
9801 ?           00:00:00 (SD-PAM)
```

Practica esto → TD4 · 3

§4 Tuberías de N comandos

Para $c_1 \mid c_2 \mid \dots \mid c_N$ el shell crea $N - 1$ tuberías y N hijos. El algoritmo cabe en un bucle con una variable `in` que guarda el extremo de lectura de la tubería anterior: para cada comando, si no es el último, `pipe`; `fork`; el hijo hace `dup2(in, 0)` y `dup2(fds[1], 1)`, cierra todo lo demás y hace `exec`; el padre cierra `in` y `fds[1]` y guarda `fds[0]` como el nuevo `in`. Al final, espera a

todos y devuelve el estado del **último** (en bash, `set -o pipefail` devuelve el del último que falló).

La regla de oro: **cada tubería debe terminar con exactamente un descriptor de escritura y uno de lectura**, en los dos procesos que la usan. Un solo `close` olvidado —en el padre o en un hijo que heredó la tubería— deja un escritor de más y el último comando espera un fin de archivo que nunca llega.

C tds/td4/pipeline_args.c

```
int in = STDIN_FILENO;
pid_t last = -1;
for (int i = 0; i < ncommands; i++) {
    int fds[2] = { -1, -1 };
    if (i < ncommands - 1)
        exit_if(pipe(fds) == -1, "pipe");

    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        if (in != STDIN_FILENO) {
            exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
            exit_if(close(in) == -1, "close");
        }
        if (fds[1] != -1) {
            exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
            exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}
```

Salida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Salida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Practica esto → A5.1 construir una tubería · TD4 · 4 y 5

§5 Más allá

- `pipe2(fds, O_CLOEXEC)` crea la tubería con los descriptores marcados para cerrarse en `exec`: evita que un programa lanzado herede extremos que no le corresponden (y que, como en §4, impidan un fin de archivo).
- `splice` y `tee` mueven datos entre una tubería y un archivo o socket dentro del núcleo, sin copiarlos al espacio del proceso.
- **Sustitución de procesos** en bash (`diff <(ls a) <(ls b)`): el shell crea tuberías y las pasa como rutas `/dev/fd/N`.
- Para comunicación **bidireccional** hacen falta dos tuberías o un `socketpair` (unidad 8).